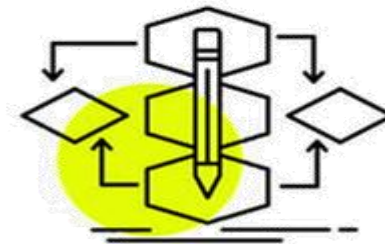


Partie 1: Raisonnement par Algorithme

CHAPITRE 3

Structures de Contrôle



Module : Informatique 1 : Algorithmique 1/Python

Pr. Mariya OUAISSA
E-mail : m.ouaissa@uca.ac.ma
Département : Informatique FSSM

Année Universitaire : 2023/2024

PLAN

1

Introduction à l'algorithmique

2

Instructions élémentaires

3

Structures de contrôle

4

Les tableaux

Les Structures de contrôle

Une structure de contrôle sert à contrôler l'exécution d'une instruction ou d'un bloc d'instructions. Deux types de structures de contrôles:

- **Structure conditionnelle** si l'exécution de l'instruction ou du bloc dépend d'une condition
- **Structure répétitive(répétitive ou boucle)** si l'exécution de l'instruction ou du bloc peut être répétée plusieurs fois (basée aussi sur une condition).

Les boucles



Situation



- Donner au moins deux versions d'un programme qui permet d'afficher le mot FSSM 10 fois.

#Version 2

```
Pour i ← 1 à 10 faire  
    Ecrire('FSSM')  
Fin Pour
```

#Version 1

```
Ecrire('FSSM')  
Ecrire('FSSM')  
Ecrire('FSSM')  
Ecrire('FSSM')  
Ecrire('FSSM')  
Ecrire('FSSM')  
Ecrire('FSSM')  
Ecrire('FSSM')  
Ecrire('FSSM')  
Ecrire('FSSM')
```

Structures répétitives

- La **structure répétitive (Boucle)** permet d'exécuter plusieurs fois une séquence d'instructions.
- Dans une boucle, le nombre de répétitions peut être connu, **fixé à l'avance**, comme il peut **dépendre d'une condition** permettant l'arrêt et la sortie de la boucle.
- Le passage dans une boucle s'appelle **itération**.

Boucle **Pour**

Nombre de
répétitions
Connu

Boucle **Tant que ... faire**

Nombre de
répétitions
Dépend d'une
condition

Boucle **Répéter... jusqu'à**

La Boucle pour

Syntaxe :

Pour I ← N à M pas 1 **Faire**
 Instructions
Fin Pour

I : variable

N : valeur initiale

M : valeur finale

Le pas de variation égale à 1

Structures de contrôle- Structures répétitives

Pour var_compteur $\leftarrow$ début à fin faire

Instr1;

Instr2;

..

Finpour

Ce bloc sera exécuté fin-début+1 fois

- **Fonctionnement**

- On affecte à var_compteur la valeur de début, puis on exécute les instructions.
- Ensuite, on incrémente var_compteur et on vérifie si sa valeur est $\leq$ de celle de fin.
- Si var_compteur $\leq$ fin, on exécute les instructions et on passe)
- On quitte cette boucle lorsque var_compteur $>$ fin.

var_compteur, début et fin doivent avoir le même type ou des types compatibles.

Algorithme : Table de multiplication de 7

Algorithme table_7

Variables

M, i : Entier

Début

Pour i ← 0 à 10 pas 1 Faire

M ← i * 7

Ecrire (i, " * 7 = ", M)

Fin Pour

Fin

**Solution avec une
boucle**

Algorithme table_7

Variables

M : Entier

Début

M ← 0 * 7

Ecrire ("0 * 7 = ", M)

M ← 1 * 7

Ecrire ("1 * 7 = ", M)

M ← 2 * 7

Ecrire ("2 * 7 = ", M)

M ← 3 * 7

Ecrire ("3 * 7 = ", M)

M ← 4 * 7

Ecrire ("4 * 7 = ", M)

M ← 5 * 7

Ecrire ("5 * 7 = ", M)

M ← 6 * 7

Ecrire ("6 * 7 = ", M)

M ← 7 * 7

Ecrire ("7 * 7 = ", M)

M ← 8 * 7

Ecrire ("8 * 7 = ", M)

M ← 9 * 7

Ecrire ("9 * 7 = ", M)

M ← 10 * 7

Ecrire ("10 * 7 = ", M)

Fin

**Solution sans utiliser
les boucles**



Table de multiplication de 7

$0 * 7 = 0$

$6 * 7 = 42$

$1 * 7 = 7$

$7 * 7 = 49$

$2 * 7 = 14$

$8 * 7 = 56$

$3 * 7 = 21$

$9 * 7 = 63$

$4 * 7 = 28$

$10 * 7 = 70$

$5 * 7 = 35$

Exercice : La boucle Pour

Ecrire un algorithme qui permet de calculer la somme des 20 premiers entiers positifs.

Algorithme Somme

Variables

S, I : Entier

Début

S ← 0

Pour I ← 1 à 20 pas 1 Faire

S ← S + I

Fin Pour

Ecrire (" La somme des 20 premiers entiers positifs est : " , S)

Fin

I		S	
I		S	0
I	1	S	1
I	2	S	3
I	3	S	6
I	4	S	10
I	5	S	15
I	...	S	...
I	19	S	190
I	20	S	210

I=1	et	Somme = 1
I=2	et	Somme = 3
I=3	et	Somme = 6
I=4	et	Somme = 10
I=5	et	Somme = 15
I=6	et	Somme = 21
I=7	et	Somme = 28
I=8	et	Somme = 36
I=9	et	Somme = 45
I=10	et	Somme = 55
I=11	et	Somme = 66
I=12	et	Somme = 78
I=13	et	Somme = 91
I=14	et	Somme = 105
I=15	et	Somme = 120
I=16	et	Somme = 136
I=17	et	Somme = 153
I=18	et	Somme = 171
I=19	et	Somme = 190
I=20	et	Somme = 210

La somme est 210

Structures de contrôle- Structures répétitives

- La boucle : **Tantque(condition)**

Tantque (condition) **faire**

Instr1;

Instr2;

...

Fin tantque

- **Fonctionnement**
- Si la condition est vraie, on exécute les instructions et on passe vérifier la condition de nouveau.
- Si elle est toujours vraie, on exécute les instructions et ainsi de suite. Ce processus se répète jusqu'à ce que la condition soit vraie.
- Si la condition est fausse à l'entrée de la boucle, le bloc ne sera jamais exécuté.



En général, les instructions doivent contenir une instruction qui **assure la sortie de la boucle** (rend la condition fausse après un certain nombre fini de répétitions).

Structures de contrôle- Structures répétitives

- La boucle : **Tantque(condition)**

Exemples :

- 1) Écrire un algorithme qui calcule la somme $1+2+3+\dots+n$. n à lire au clavier.
- 2) Écrire un algorithme qui affiche les diviseurs d'un nombre lu au clavier. Utiliser l'opérateur %.



```
Algorithme Somme;  
Var s,n, i : entier;  
Début  
     $s \leftarrow 0;$   
     $i \leftarrow 1;$   
    lire(n);  
    Tantque ( $i \leq n$ ) faire  
         $s \leftarrow s+i;$   
         $i \leftarrow i+1;$   
    fin tantque;  
    écrire(s);  
fin.
```

7-Structures de contrôle- Structures répétitives

- La boucle : **Tantque(condition)**

Exercice :

- 1) Écrire un algorithme qui calcule la somme $1+2+3+\dots+n$. n à lire au clavier.
- 2) Écrire un algorithme qui affiche les diviseurs d'un nombre lu au clavier. Utiliser l'opérateur %.



```
Algorithme Diviseurs;  
Variables  
  n, i:entiers;  
Début  
  Lire(n);  
  i ← 1;  
  Tantque i ≤ n faire  
    Si (n%i)=0 alors  
      Ecrire( i);  
    Finsi  
    i ← i+1;  
  Fin tantque  
Fin .
```


Structures de contrôle- Structures répétitives

La structure :
répéter ... jusqu'à(condition)

répéter

Instr1;

Instr2;

...

Jusqu'à (condition)

Syntaxe :

REPETER

<actions simples>

JUSQU'A <condition>

Fonctionnement :

1. on exécute le corps

2. on évalue la condition , puis

● si la condition est **vraie** : on quitte le **REPETER**

● si la condition est **fausse** on **recommence**



En général, les instrs doivent contenir une instruction qui assure la sortie de la boucle: rendre la condition fausse après un certain nombre fini d'itérations ou autre .

→ Les instructions contrôlées par cette structure sont exécutées au moins une fois

Structures de contrôle- Structures répétitives

Exemple

répéter ... jusqu'à(condition)

Algorithme Somme;

Var s,n, i : entier;

Début

s $\leftarrow$ 0;

i $\leftarrow$ 1;

lire(n);

Répéter

s $\leftarrow$ *s*+*i*

i $\leftarrow$ *i*+1

Jusqu'à (*i*≤*n*)

ecrire(s);

fin.

Exemples Les Répétitions ou Boucles

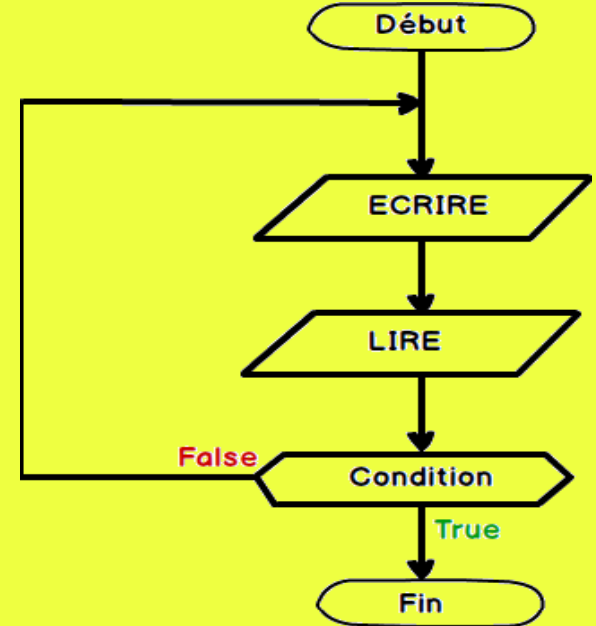
Concept	Algorithmique textuelle
Boucles pour	Pour i de 1 à 10 faire Écrire(7 x i) Fin pour
Boucles tant que	i ← 1 Tant que i ≤ 10 faire Écrire(7 x i) i ← i + 1 Fin tant que
Boucles répéter jusqu'à	i ← 1 Répéter Écrire(7 x i) i ← i + 1 Jusqu'à i > 10
Boucles imbriquées	Pour i de 1 à 5 faire Pour j de 1 à 5 faire Écrire(i x j) Fin pour Fin pour

Un Algorithme peut
s'écrire aussi en
représentation
Graphique



Organigramme

Chaque instruction et chaque
structure de contrôle possède
une représentation graphique.



Algorithme par Organigramme

Affectation

Lecture

Ecriture

Début

Fin

Séquence

Reste de l'algorithme

`surface ← PI*rayon*rayon`

`Lire(rayon)`

`Ecrire(surface)`

Début

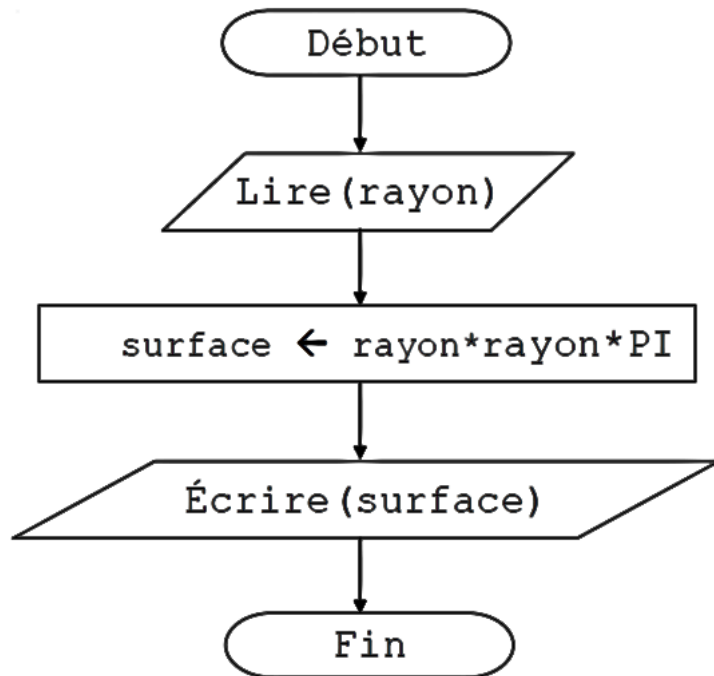
Fin



(avant ou après l'instruction en cours)

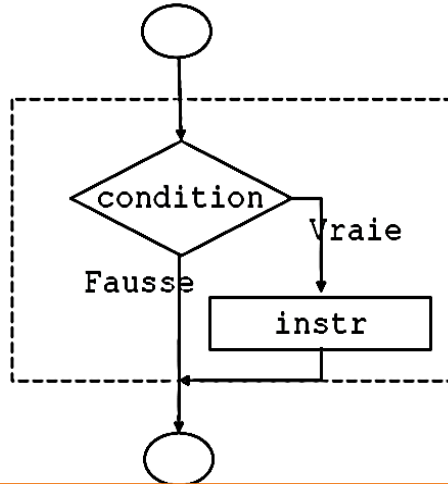
Algorithme par Organigramme

- Exemple 1

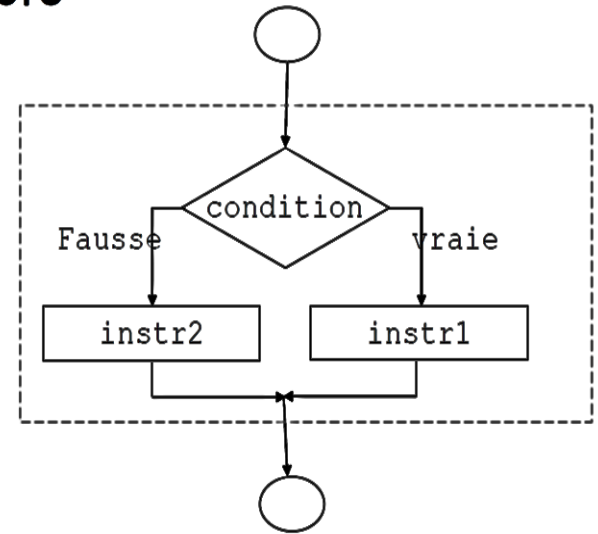


Organigramme

Si (condition) alors
instr;
Finsi

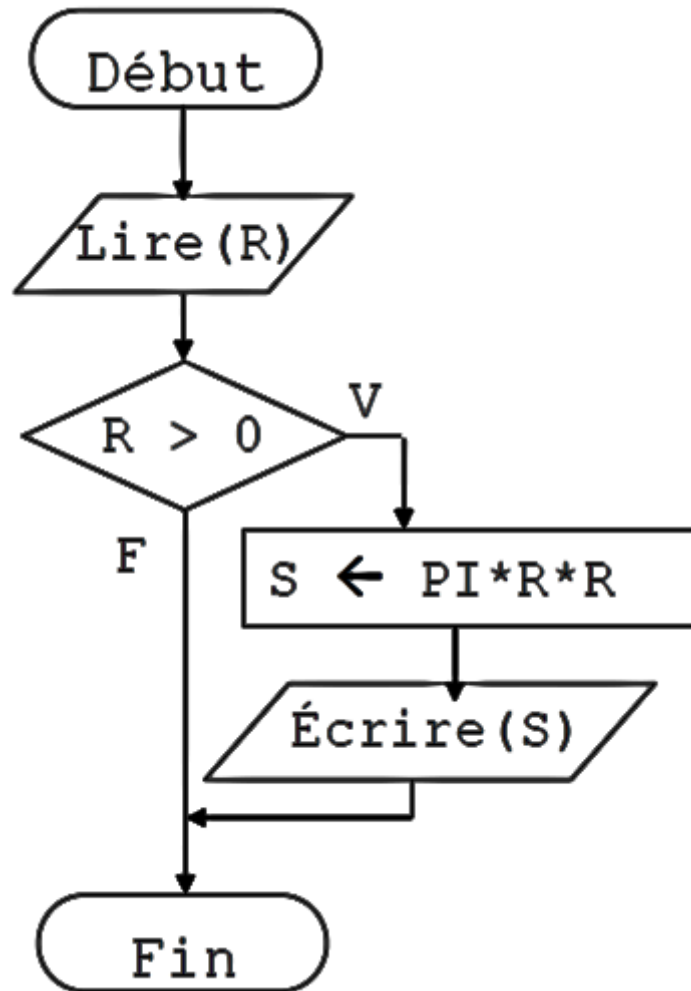


Si (condition) alors
instr1;
Sinon
instr2;
Finsi



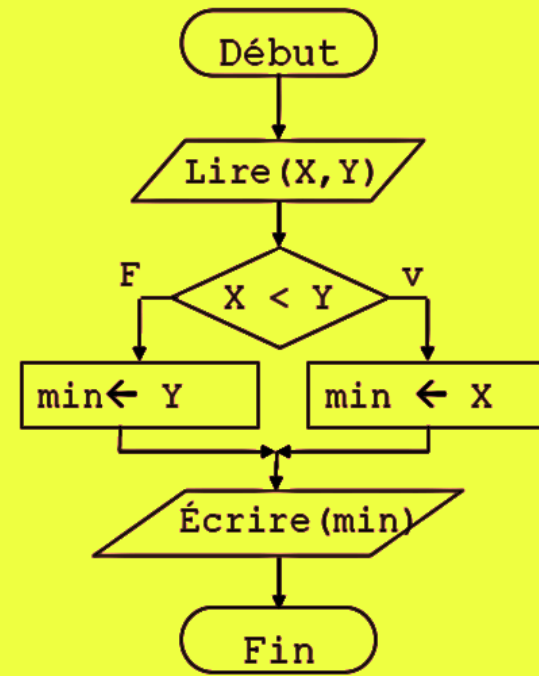
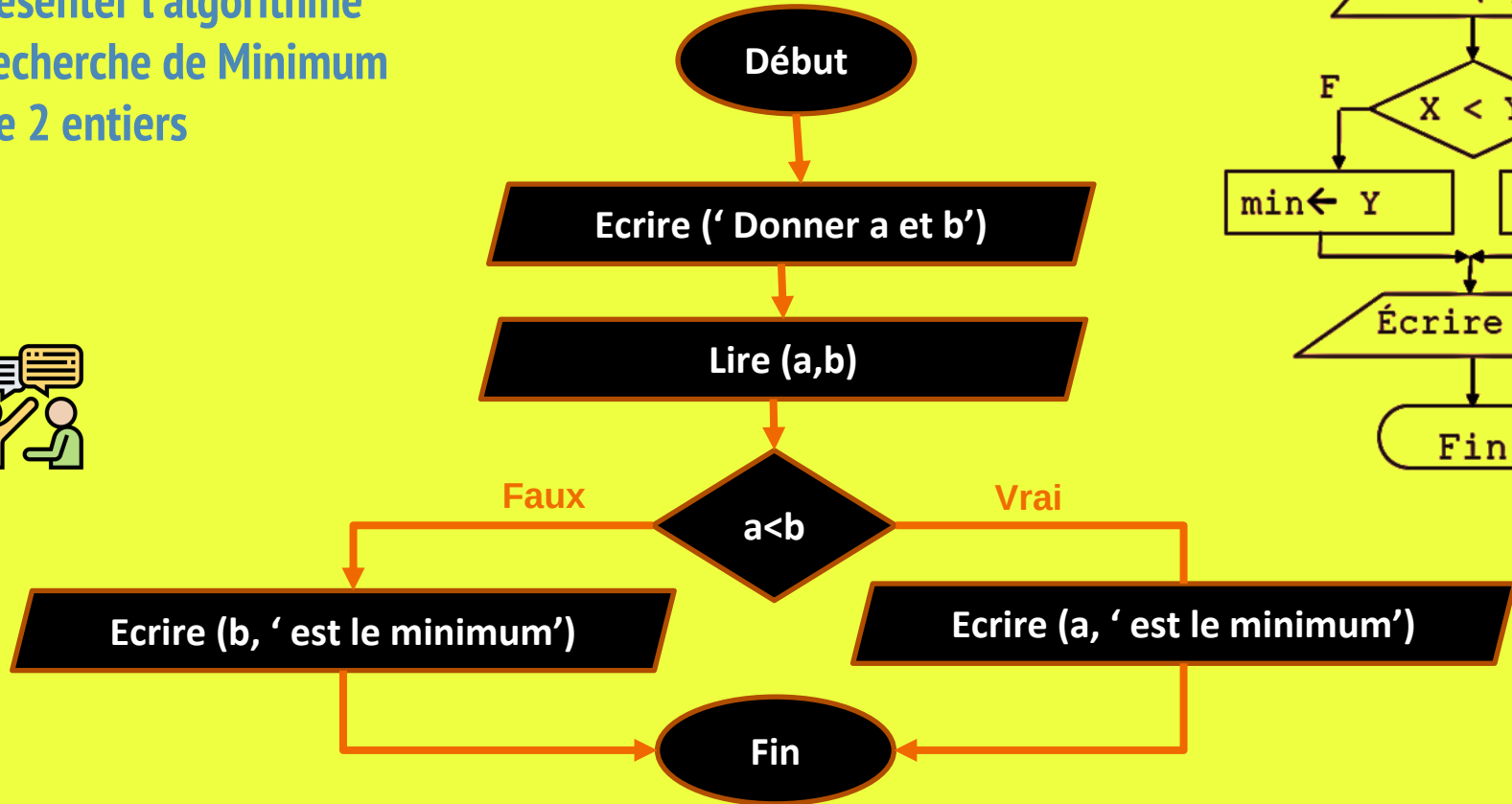
Organigramme

- Exemple 3



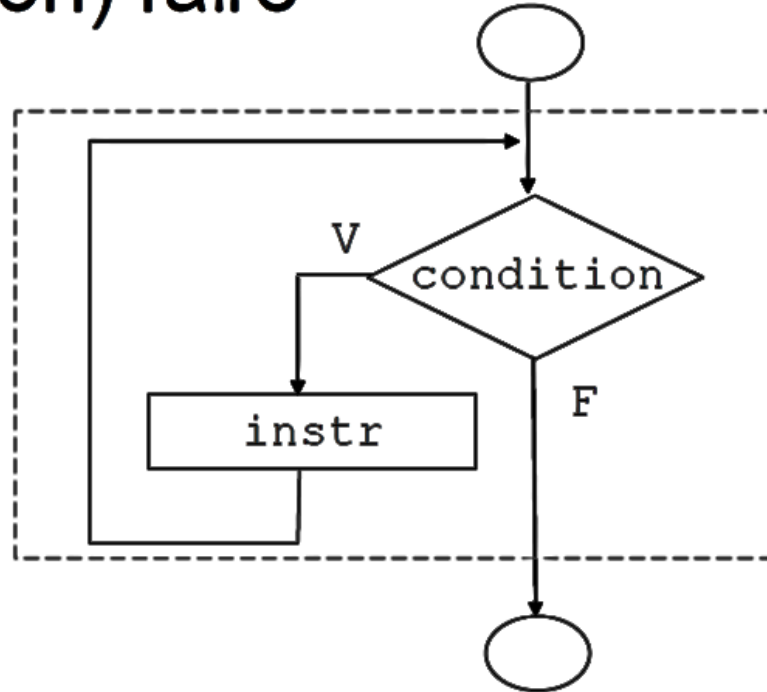
Exercice

représenter l'algorithme
de recherche de Minimum
entre 2 entiers



Organigramme – Boucle

Tantque (condition) faire
instr;
Fin tantque



Organigramme – Boucle Pour

Algorithme exemple_5

Variables

i, n : entier

Début

Lire (n)

$S \leftarrow 0$

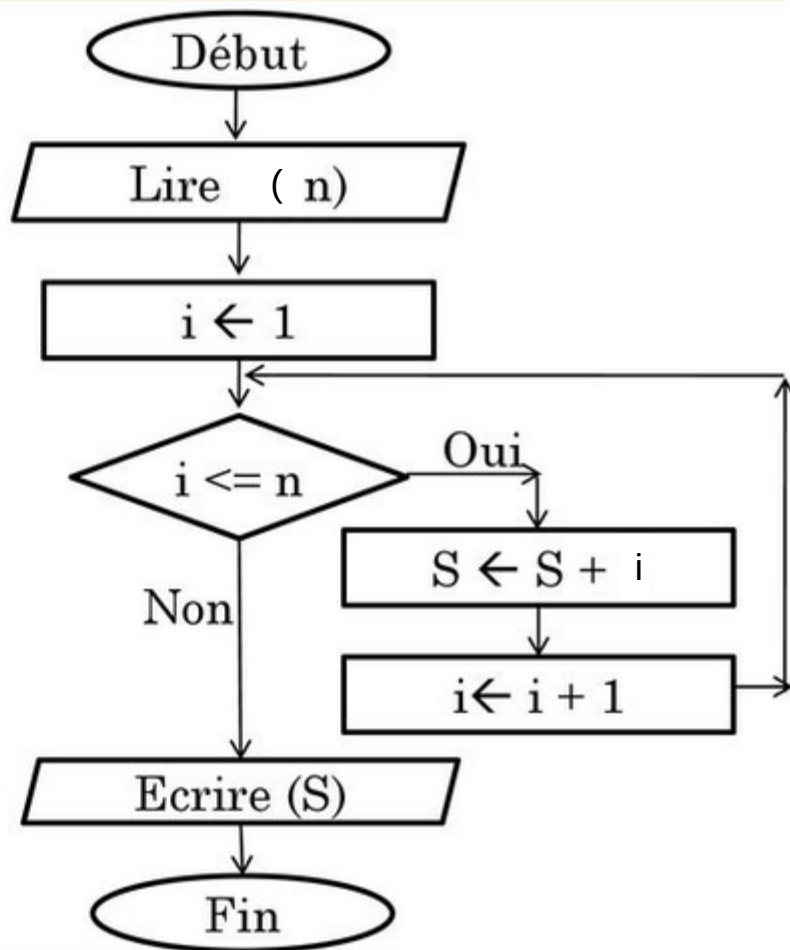
Pour $i \leftarrow 1$ **à** n **faire**

$S \leftarrow S + i$;

Fin-Pour

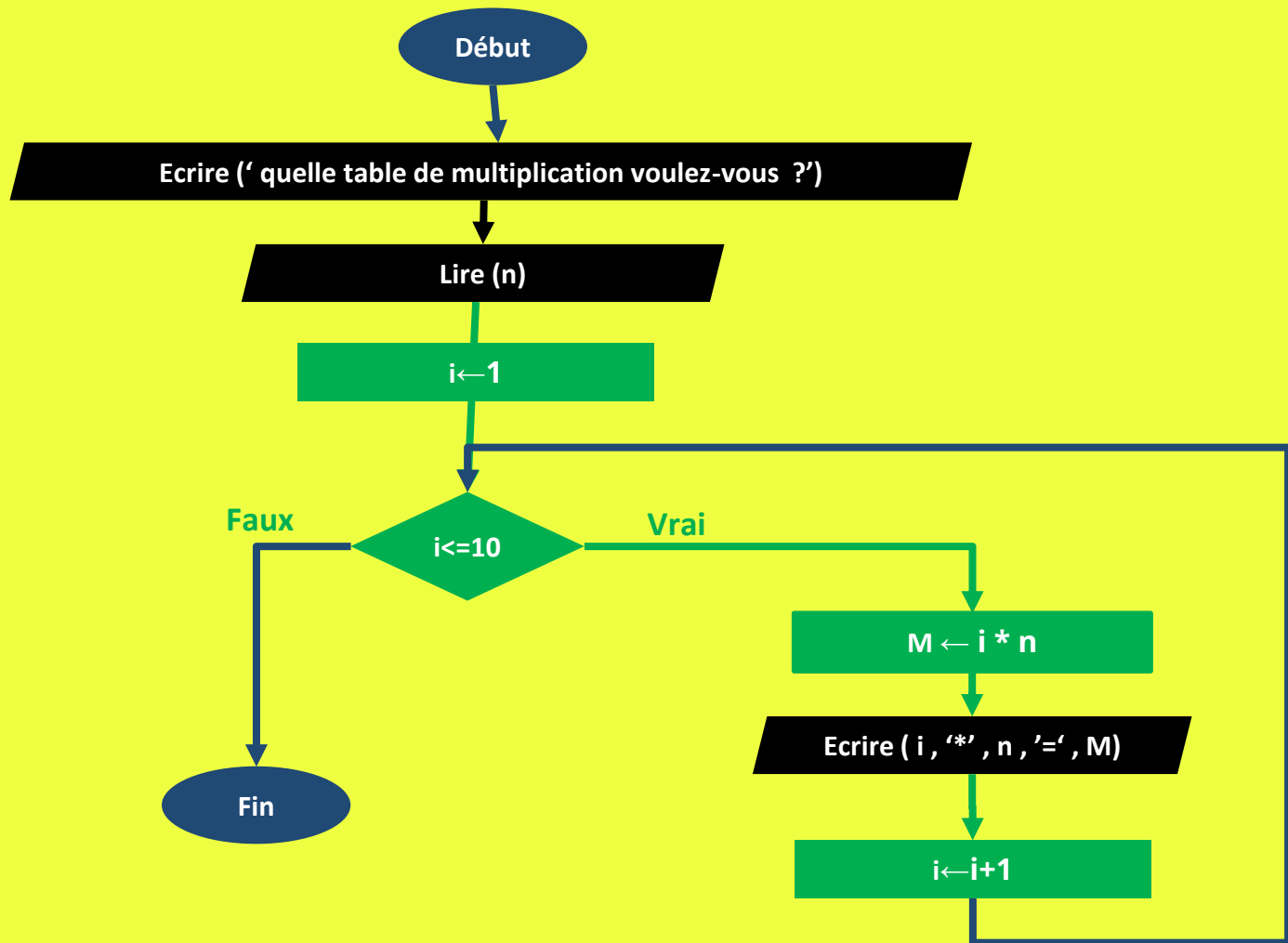
Ecrire (S)

Fin



Exercices:

- 1- Représenter l'algorithme qui demande à l'utilisateur de saisir un nombre et afficher sa table de multiplication.**
- 2- Représenter l'algorithme qui lit la moyenne d'un étudiant et affiche la mention associée.**



Structures de contrôle -résumé

Il y a trois structures principales de contrôle qui permettent de construire des algorithmes

Bloc d'instruction	Alternative	Répétition (Boucle)
début <i>instruction1</i> <i>instruction2</i> fin	Alternative simple si <i>ExpressionBooléenne</i> alors <i>BlocInstruction1</i> sinon <i>BlocInstruction2</i> finsi;	tant que <i>ExpressionBooléenne</i> faire <i>BlocInstruction</i> fantantque;
	Altérnative multiple selon variable que <i>cas1</i> : <i>BlocInstruction1</i> <i>cas2</i> : <i>BlocInstruction2</i> autre : <i>BlocInstruction</i> Finselon	le bloc d'instruction peut ne pas être exécuté et il y a une variable indicatrice pour <i>VariableIndicatrice</i> allant de <i>ValeurInitiale</i> à <i>ValeurFinale</i> par pas de <i>ValeurPas</i> faire <i>BlocInstruction</i> finpour;
		le bloc d'instruction est exécuté au moins une fois (ne se traduit pas en python) répéter <i>BlocInstruction</i> jusqu'à <i>ExpressionBooléenne</i> finrépéter;